

UNIT- 1

Q1. a.

Aspect	CBOW (Continuous bag of words)	Skip-gram
Training Objective	This technique helps in predicting the target word using the surrounding context words.	Whereas skip-gram predicts the surrounding context words from a given target word.
Computational Complexity	Faster, because it takes an average of the context vectors & perform fewer predictions. Hence it is suitable for large datasets with limited resources.	Perform better on rare words since each word is independently trained against surrounding contexts.
Performance on rare words	Perform less effectively on rare words because averaging out context may dilute unique patterns.	Perform better on rare words since each word is trained independently against surrounding contexts.
Suitability for Domain specific corpora	Works well with frequent words. It performs better for general-purpose corpora. Also where speed is important - it is useful.	Much better for domain-specific corpora, where rare terms are used & is important e.g. medical, law etc.

②

↳ For the analogy :

king - man + woman $\approx$ queen, each word is represented by its embedding vector:

$$\vec{k} - \vec{m} + \vec{w} \approx \vec{q}, \text{ where}$$

$\vec{k}$ = vector representation of king
 $\vec{m}$ = " " " man
 $\vec{w}$ = " " " woman
 $\vec{q}$ = " " " queen

The operation does the following :

1. $\vec{k} - \vec{m}$ removes the "male" component from king
2. $\vec{w}$ addition results in introduction of "female" component.
3. The vector output then becomes close to queen.

Mathematically, $\vec{r} = \vec{k} - \vec{m} + \vec{w}$, the model

look-up for the word vector most similar to $\vec{r}$.

This similarity is measured using cosine similarity:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}, \text{ where :}$$

$A \cdot B$ = dot product of vectors

$\|A\| \|B\|$ = are the magnitudes.

θ = angle between A & B vectors.

A cosine similarity value closer to 1 means the vectors are semantically closer to each other/ are very similar.

③

→ why cosine similarity is preferred over Euclidean distance?

Since cosine similarity focuses on semantic orientation or direction between 2 vectors it is more helpful in embedding space. Euclidean distance focuses more on magnitude (straight-line distance b/w vectors) so it's not suitable as it's very sensitive to vector magnitude. Large vector magnitudes can distort similarity.

Ex. Two vectors may have different magnitudes,

$(200, 200)$; $(10, 10)$

- ↳ Euclidean distance says they are apart, but
- ↳ Cosine similarity says they are highly similar since they both point in the same dirⁿ.

Explanation.

Q1.5. Point.

Problem :

Glove embeddings trained on Wikipedia understand general meanings, but not product-specific terms eg. DSLR, mirrorless, or APS-C.

Reason :

Wikipedia may treat these domain specific terms as rare or generic words, while in e-commerce they hold very different meanings.

Strategy :

Use continued training / fine-tuning of pre-trained GloVe embeddings on an e-commerce product corpus.

④

<u>Point.</u>	<u>Explanation.</u>
Corpus required :	Product titles, descriptions, reviews, search queries, category names etc.

Steps to Adapt :	Initialize the embedding matrix with pre-trained GloVe vectors, then train further on domain specific text instead of training from random initialization. ∴ common words will retain general meaning, while domain words learn product-specific relationships.
------------------	---

Impact of corpus size, window size & dimensionality?

<u>Factor</u>	<u>Impact on embedding quality.</u>
Corpus size	Larger the domain corpus better the embeddings because model can can see more product contexts. Small corpora may result in poor or unstable vectors.
Window size	Small window captures syntactic/local relations, eg. "APS - c sensors" while large windows captures broader semantic relations.
Dimensionality	Higher dimensions capture richer meanings but computation & data-set required is huge. Too low may lose meaning.

5

Expected Empirical trends:

Setting

Small corpus & high dimension

Small window

Large window

Large corpus & moderate/high dimension

Fine-tuned GloVe

Expected Result

Overfitting & noise.

Better for technical product phrases.

Better for category-level similarity.

Better semantic quality.

Better than original wikipedia GloVe for product terms.

Q1. c.

Dimension

Definition

Algorithm

Output Quality

Speed

Stemming

Reduces words to their root form by removing any prefixes/suffixes.

Rule-based chopping.
Ex. Porter Stemmer or Snowball stemmer.

May produce non-meaningful roots.

Faster, since it uses simple rules.

Lemmatization

Converts words to their meaningful dictionary base form (lemma).

Uses vocabulary, morphological analysis, & POS (part-of-speech) information.

Produces valid dictionary words.

Slower, as it requires linguistic analysis.

⑥

<u>Dimension</u>	<u>Stemming</u>	<u>lemmatization</u>
Accuracy	less accurate semantically	more semantically accurate.
language grasp	Doesn't consider grammar / context	Considers grammar & word meaning.

⑦ Applying both techniques :

<u>Original word</u>	<u>Stemming output</u>	<u>lemmatization output</u>
running	run	run
better	better	good
studies	Studi	study
happily	happili	happily
wolves	wolv	wolf.

⑧ Lemmatization is preferred when semantic meaning & grammatical correctness are important.

Suitable NLP scenarios :

- a) Chatbots b) Sentiment analysis c) Search systems
- d) Q & A systems e) Legal / medical NLP systems.

Real - world Example :

"patients running high fever"

with stemming : "running" → "run", it may distort meaning or create unnatural forms.

with lemmatization : "running" → "run", words remain linguistically similar & context-aware.

②

Q4) a. Step-by-step Naïve Bayes sentiment classifier:

1. Data collection

Collect & label customer reviews:

Review	Sentiment
"Great product & fast delivery"	Positive
"OK OK product"	Neutral
"Bad experience with product & delivery"	Negative.

These labelled review become the training dataset.

2. Text preprocessing

Before training, clean the text data

Common steps:

- convert text to lowercase
- Remove punctuations
- Remove stopwords
- Tokenization
- Stemming / Lemmatization.

Example:

"The product was running smoothly" — original review

"product run smooth" — post pre-processing.

This step reduces noise & improve model quality.

3. Feature extraction (TF-IDF)

ML models can't directly understand text, so convert text into numerical vectors using: TF-IDF.

TF-IDF score is: $TF-IDF(t, d) = TF(t, d) \times IDF(t)$

8

Example :

Word
excellent
product
the

TF-IDF importance

high
medium
very low.

Common words receive lower importance.

4. Train - test split.

Split the dataset into :

Training set $\rightarrow$ 80% (400 reviews)

Testing set $\rightarrow$ 20% (100 reviews)

Training data teaches the model

Test data evaluates performance.

5. Model training using Naïve Bayes

Train the Naïve Bayes classifier on TF-IDF vectors.

It uses probability : $P(\text{class}|\text{Text})$

It predicts the class with highest probability.

6. Prediction

The trained model predicts sentiment for unseen reviews.

7. Model Evaluation

Use the following techniques/parameters to evaluate model accuracy :

a) Accuracy b) Precision c) Recall

d) F1-score e) Confusion matrix.

⑨

⑥ Precision, Recall & F1-score

① Precision measure "Out of all predicted positive reviews, how many were actually positive?"

$$\text{Precision} = \frac{TP}{TP + FP} \quad \begin{cases} TP: \text{true +ve} \\ FP: \text{false +ve} \end{cases}$$

High precision means fewer false alarms.

② Recall measure "Out of all actual +ve reviews, how many were correctly detected?"

$$\text{Recall} = \frac{TP}{TP + FN} \quad \begin{cases} TP: \text{true +ve} \\ FN: \text{false -ve} \end{cases}$$

High recall means fewer missed +ve reviews.

③ F1-score (Harmonic mean of Precision & Recall)

$$F1 = 2 \times \frac{(\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})}$$

It balances both metrics together.

⇒ F1-score is preferred because :

- a) It considers both false +ve & false -ves.
- b) Gives a balanced evaluation
- c) Works better for imbalanced sentiment datasets.

(10)

Q4. b)

Topic	Top words	Label	Justification.
Topic 1	goal, match, player, stadium, coach, score, league.	Sports/Football	These words are strongly related to sports events, teams, matches & competitions.
Topic 2	Budget, tax, parliament, policy, election, vote	Politics/ Government.	These words are related to politics, govt. decisions, public-policy & law-making.
Topic 3	Vaccine, hospital, patient, disease, drug, treatment, doctor.	Healthcare/ Medicine	These words are connected to hospitals, patients, disease & medical treatments.

- ① A high coherence score means the words inside a topic are strongly related to each other & the topic is easy for humans to interpret.
- ② If topic 2 also contained "goal" & "score" that would mean the politics topics are getting mixed with sports terms. In that case I would recommend:
 - ① Better preprocessing : remove irrelevant or noisy words from articles.
 - ② Stopword/domain-specific filtering : Add words like "goal" & "score" to a custom stopwords list if they are not useful for political topics.

11

- ③ Improve tokenization : keep meaningful phrases like "tax policy", "electoral vote", "football score", etc.
- ④ Remove mixed-topic : Articles covering both politics, & sports may confuse the model.

24)c. NER pipeline for financial news.

NER is used to automatically identify entities such as:
a) People b) organizations c) locations d) Currency etc.
- from text documents.

NER pipeline :

- ① Data collection : collect financial news articles from newspapers, APIs or websites.
- ② Preprocessing (text) : Convert text to lowercase, remove noise, punctuation handling etc.
- ③ Tokenization : Split sentence into words/tokens
- ④ POS tagging : Identify grammatical roles such as noun, verb, adjectives.
- ⑤ Feature Extraction : Extract useful features like prefixes, suffixes, neighbouring words, word-embeddings etc.
- ⑥ Sequence Labeling : Apply HMM, CRF, or neural models to assign entity labels.
- ⑦ Entity Extraction : Extract final entities with tags like person, org, location.
- ⑧ Evaluation : Evaluate using Precision, recall, F1-score.

⑩

⑫

① Sequence labeling in NER

NER is treated as a sequence labelling problem because each word in a sentence receive a label.

Example :

word	Label
Apple	ORG
acquired	O (not entity)
Beats	ORG
in	O (not entity)
California	LOCATION

② Role of HMM in NER

- ↳ probabilistic sequence model.
- ↳ Uses transition probabilities b/w tags.
- ↳ predicts the most probable tag sequence.
- ↳ Simple probabilistic model, works reasonably well on small datasets.

③ Role of CRF in NER

- ↳ CRF improves over HMM.
- ↳ CRF models the conditional probability, $P(Y|X)$, where $Y \Rightarrow$ output labels
 $X \Rightarrow$ input words.
- ↳ captures contextual dependencies
- ↳ Requires feature engineering & training can be slower.

④ Role of neural approaches in NER.

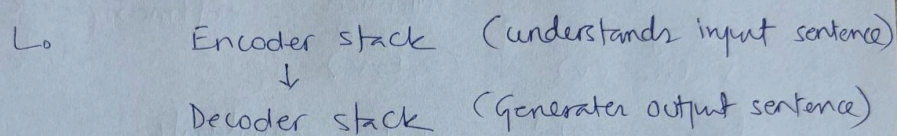
Modern NER systems use deep learning models such as :

- a) RNN b) LSTM c) BiLSTM d) BERT (transformer model)

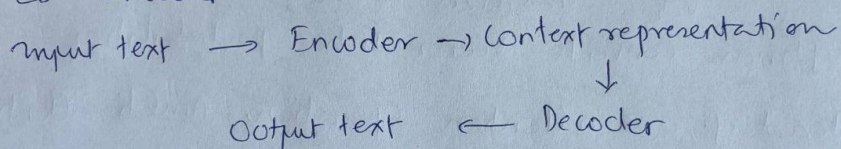
Advantages of Neural Approacher :

- a) Automatic feature learning.
- b) Better contextual understanding
- c) Handles long dependencies.
- d) Higher accuracy.

Q6). a). Architecture of the Transformer Model :



↳ Encoder-Decoder Flow



↳ Encoder stack : Each encoder layer contains :

- ① Multi-head self-attention
- ② Add & layer normalization
- ③ Feed forward network
- ④ Add & layer normalization.

(Usually repeated multiple times)

↳ Decoder stack : Each decoder layer contains :

- ① masked Multi-head self-attention
- ② Encoder-decoder attention
- ③ Feed forward network
- ④ Layer normalization & residual connections.

The decoder predicts output words one by one.

14

a) multi-head self attention

↳ most important component

↳ It allows each word to focus on other relevant words in the sentence.

↳ self-attention mechanism where each word is converted into :

$$\left. \begin{array}{l} \text{Query (Q)} \\ \text{Key (K)} \\ \text{Value (V)} \end{array} \right\} \begin{array}{l} \text{Attention} \\ \text{score} \\ (Q, K, V) \end{array} = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V,$$

where, QK^T computes similarity.

$\sqrt{d_k}$ stabilizes value.

Softmax converts scores into probabilities.

↳ Instead of one attention operation, multi attention heads run in parallel.

b) Position-wise Feed Forward Network (FFN)

After attention, each token passes through a small neural network independently.

input $\rightarrow$ linear $\rightarrow$ ReLU $\rightarrow$ linear

c) Positional Encoding

↳ Transformer process words in parallel, so they don't naturally understand word order.

↳ Positional encoding adds sequence order information.

d) Residual connections & layer normalization

↳ These components stabilize & improve deep network training.

↳ Residual connections prevents vanishing gradients

(15)

↳ layer normalization helps stabilize training, reduces internal covariate shift & improves convergence speed.

↳ for every sublayer:

input $\rightarrow$ sublayer $\rightarrow$ Residual addition $\rightarrow$ layer normalization.

Q6). b.

Aspect	BERT	GPT
Architecture Type	Encoder-only transformer architecture (language understanding)	Decoder-only transformer architecture. (text generation)
Pre-training Objective	masked language modeling (MLM) or Next sentence prediction (NSP)	Auto-regressive language modeling (predict next word)
Directionality of attention	Bidirectional attention (looks both left & right contexts)	Unidirectional / causal attention (looks only left-context)
Best suited downstream tasks	Classification, sentiment analysis, NER, question answering. [spam detection, entity recognition]	Text generation, chatbots, summarization, story completion. [ChatGPT]

Q6) c. Key issues :

- ① Small dataset problem
 - ↳ BERT is a very large deep learning model.
 - ↳ With only 5000 samples, the model may not learn task-specific patterns effectively.
 - ↳ Meanwhile TF-IDF + logistic regression works well on a smaller dataset because of fewer parameters, lower overfitting risk & simpler optimization.
 - ② Learning Rate sensitivity
 - ↳ BERT fine-tuning is highly sensitive to learning rate
 - ↳ Too high destroys pre-trained representations, & too low, then learning becomes extremely slow.
 - ③ Overfitting
 - ↳ Training for 10 epochs on a small dataset may cause overfitting.
 - ④ Tokenization problems
 - ↳ BERT uses wordpiece tokenization
 - ↳ Using the wrong tokenizer reduces semantic quality.
- To improve performance following can be considered :
- a) Tune learning rate carefully
 - b) Reduce epochs
 - c) Improve pre-processing / tokenization
 - d) Increase dataset size or apply augmentation.

(17)

Q8). a. DoRA in LLM fine-tuning

DoRA fine-tunes large language models efficiently by freezing the original model weights & training only additional matrices.

Instead of updating the full weight matrix X ,

DoRA learns a low-rank update :

$$X' = X + BA, \text{ where}$$

$X \Rightarrow$ original frozen weights

$B, A \Rightarrow$ small low-rank trainable matrices.

① low-rank decomposition reduces parameters :

A large matrix :

$$d \times k,$$

is replaced by 2 smaller matrices,

$$d \times r \text{ and } r \times k, \text{ where } r \ll d, k.$$

This drastically reduces the :

$\rightarrow$ trainable parameters

$\rightarrow$ training costs

$\rightarrow$ GPU memory usage.

② Role of rank ' r ' :

Small r

Fewer parameters

Faster training

Lower adaptation power

Large r

Better learning capacity

Higher memory usage

Better task adaptation.

(18)

- ⊙ QLoRA combines LoRA and 4-bit quantization to fine-tune LLMs on limited hardware. It stores base model in 4-bit precision, & greatly reduce VRAM usage.

Q5.b. Interpretation of the metric discrepancy:

Metric	Observation	Implication
ROUGE-L = 0.54	High lexical overlap	Summary captures imp. content structure
BERT = 0.61 ROUGE	low semantic similarity	Context may still differ from reference
BLEU low/moderate	Weak exact n-gram matching	Model may paraphrase heavily.

- ⊙ Limitations of BLEU:

- ↳ It relies on exact n-gram overlap & brevity penalty
- ↳ semantic similarity is ignored
- ↳ valid paraphrases get unfairly penalized.

- ⊙ Why BERTScore is Better?

Since it uses contextual embeddings from BERT.

It captures:

- | | |
|-------------------------|--------------------------|
| → semantic meaning | → paraphrases |
| → contextual similarity | → synonym relationships. |

- ⊙ Composite evaluation strategy:

Use all metrics together:

metric

ROUGE - 1/2L

BLEU

Purpose

Content overlap

Fluency + exact phrasing.

- context

metric
BERTScore

purpose
Semantic correctness

Q8.c) Bias in LM means model produces unfair or ~~sto~~ stereotypical outputs due to biased training data.

Exampler :

- ① Gender bias: "Nurse" → female ; "Engineer" ⇒ male
- ② Racial bias: associating certain ethnic group with crime / poverty.
- ③ Occupational bias: "CEO" linked mostly with men.

↳ Bias mitigation techniques :

- ① Counterfactual data Augmentation
(add balanced examples by swapping attributes)
- ② RLHF (humans rank safer / fairer outputs, & the model learns preferred behaviour)

↳ Responsible AI deployment matters in domains like : healthcare, hiring, finance, law etc.

since biased output can cause :

- unfair decisions
- discrimination
- legal / ethical issues
- loss of trust.

{ Responsible AI will ensure fairness, safety & reliability. }